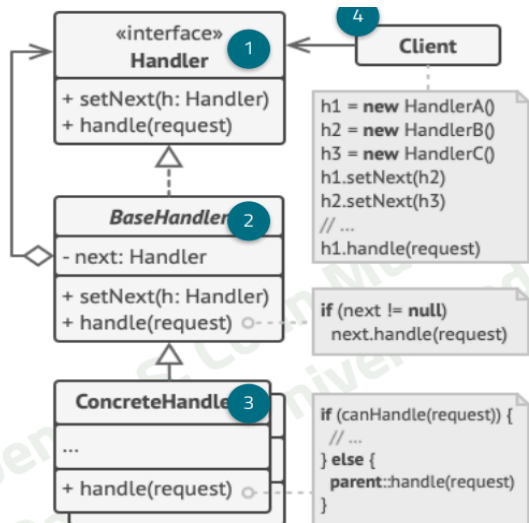
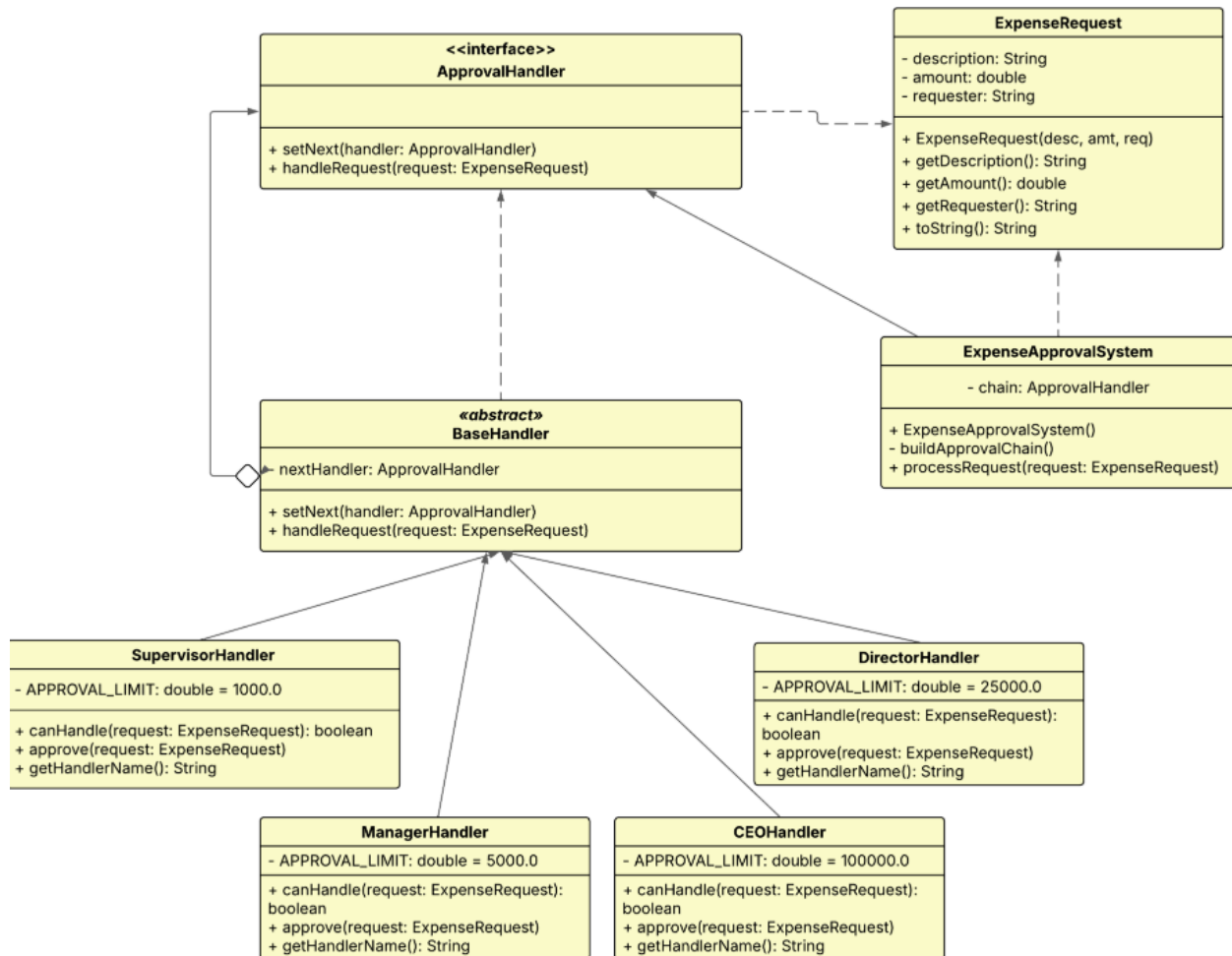
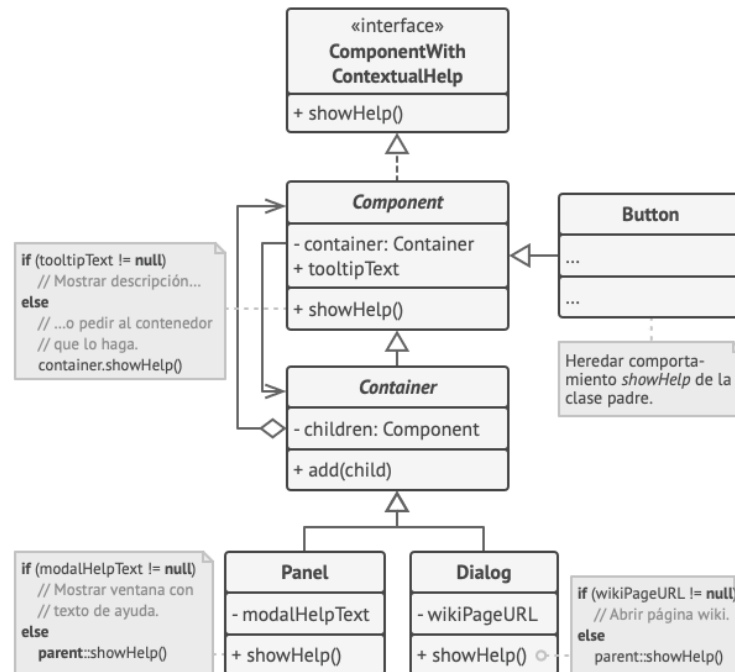


1. Chain of Responsibility (Cadena de Responsabilidad)



- 1 Interfaz para los Manejadores. Tiene un método para atender la solicitud y otro para remitirla a otro manejador.
- 2 Manejador Base Abstracto. Incluye la referencia al siguiente manejador.
- 3 Manejador Concreto contiene la lógica para atender la solicitud (decide si la atiende y/o la pasa al siguiente manejador).
- 4 Cliente arma la cadena de manejadores (estática o dinámicamente) y le envía la solicitud.





Problema que resuelve: Cuando múltiples objetos pueden manejar una solicitud y no se sabe cuál específicamente.

Cómo funciona: Crea una cadena de manejadores donde cada uno decide si procesa la solicitud o la pasa al siguiente.

Ejemplo:

```

java
// Sistema de aprobación de gastos
Manejador gerente = new Gerente();
Manejador director = new Director();
Manejador vicepresidente = new Vicepresidente();

gerente.setSiguiente(director);
director.setSiguiente(vicepresidente);

gerente.procesarSolicitud(solicitudGasto); // Se pasa por la cadena

```

Cuándo usarlo:

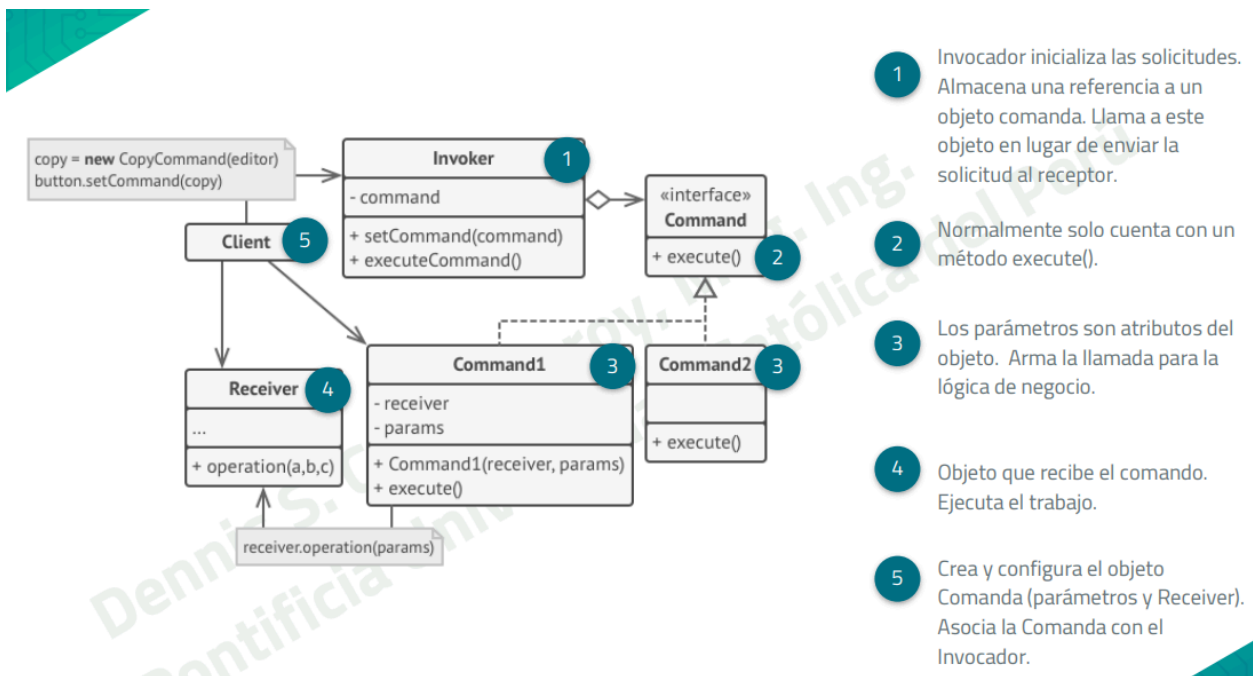
- ☒ Múltiples objetos pueden procesar una solicitud
- ☒ Quieres desacoplar el emisor de los receptores

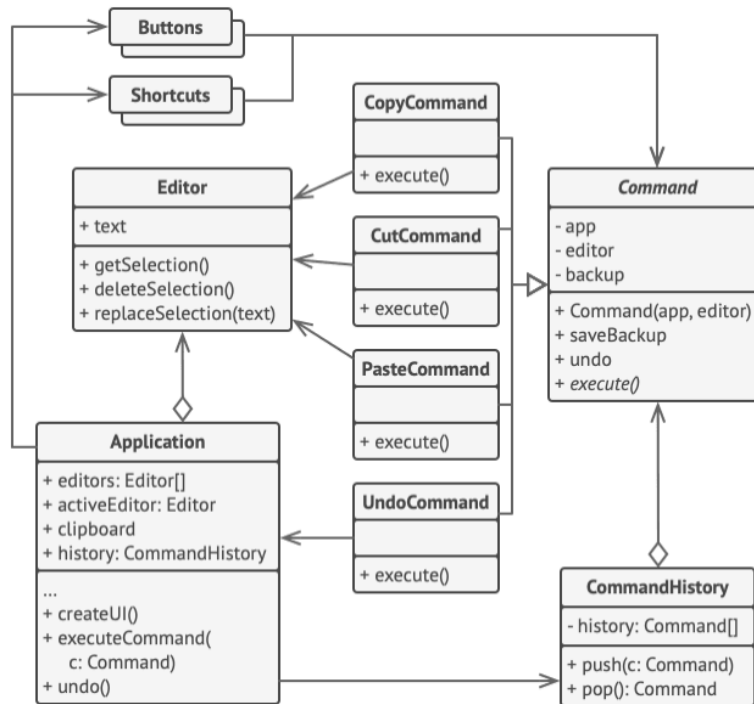
-  Necesitas determinar el manejador en tiempo de ejecución

Consecuencias:

-  Control del orden de procesamiento
-  Flexibilidad para agregar manejadores
-  Overhead por objetos que solo reenvían

2. Command (Comando)





Problema que resuelve: Necesidad de parametrizar objetos con operaciones, encolar operaciones o soportar deshacer.

Cómo funciona: Encapsula una solicitud como un objeto, permitiendo parametrizar clientes con diferentes solicitudes.

Ejemplo:

```
java
// Editor de texto con comandos deshacer
Command copiar = new CopiarCommand(editor, textoSeleccionado);
Command pegar = new PegarCommand(editor, portapapeles);
Command deshacer = new DeshacerCommand(historial);

botonCopiar.setCommand(copiar);
botonPegar.setCommand(pegar);

botonDeshacer.setCommand(deshacer);
```

Cuándo usarlo:

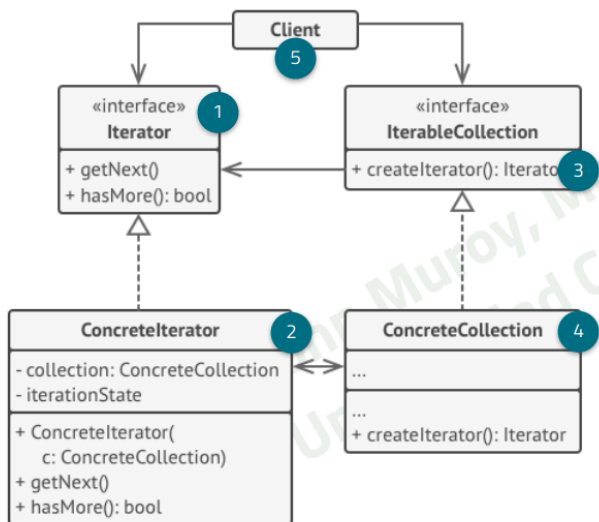
- ☒ Operaciones deshacer/rehacer

-  Operaciones en cola o programadas
-  Parametrización de operaciones

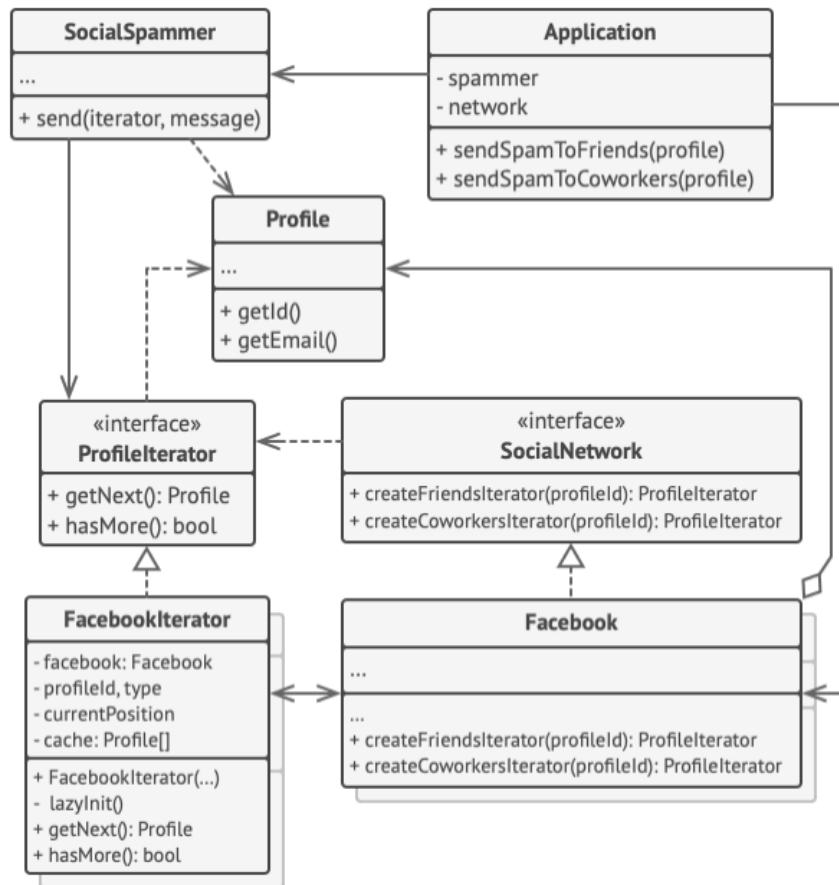
Consecuencias:

-  Desacopla invocador y receptor
-  Fácil agregar nuevos comandos
-  Complejidad adicional

3. Iterator (Iterador)



- 1 Declara los métodos para recorrer una la colección.
- 2 Implementa los métodos para recorrer la colección.
- 3 Declara el método para obtener un iterador compatible con la colección.
- 4 Implementa la colección y el método para obtener un iterador compatible.
- 5 Trabaja con el iterador y la colección a través de sus interfaces a fin de no generar acoplamiento.



Problema que resuelve: Recorrer elementos de una colección sin exponer su estructura interna.

Cómo funciona: Proporciona una interfaz para acceder secuencialmente a los elementos de una colección.

Ejemplo:

```

java
// Recorrer diferentes colecciones uniformemente
List<String> lista = Arrays.asList("A", "B", "C");
Set<Integer> conjunto = new HashSet<>(Set.of(1, 2, 3));

Iterator<String> iterLista = lista.iterator();
Iterator<Integer> iterConjunto = conjunto.iterator();

while(iterLista.hasNext()) {
    System.out.println(iterLista.next());
}

```

}

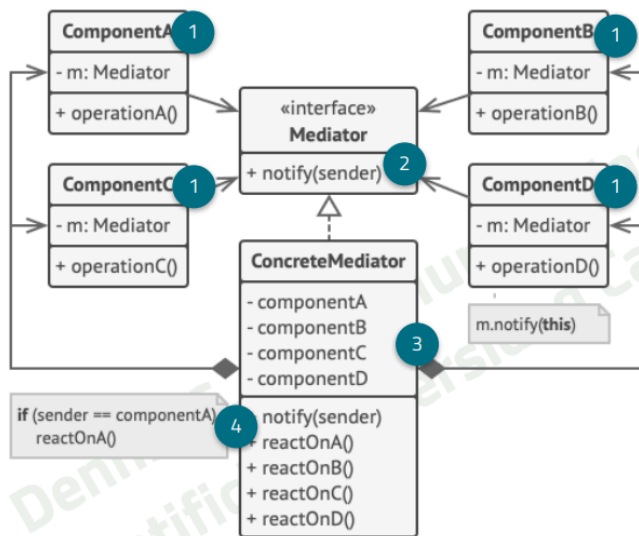
Cuándo usarlo:

- ☒ Colecciones con estructuras complejas
- ☒ Múltiples formas de recorrer una colección
- ☒ Ocultar implementación interna

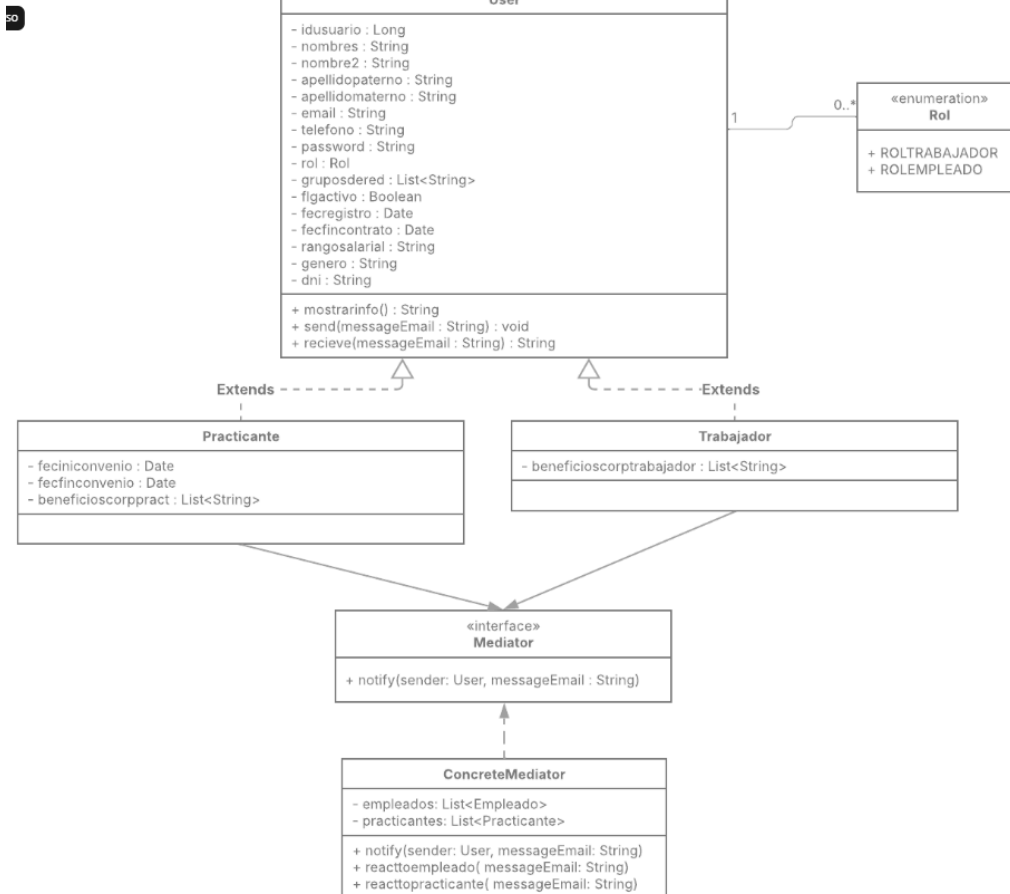
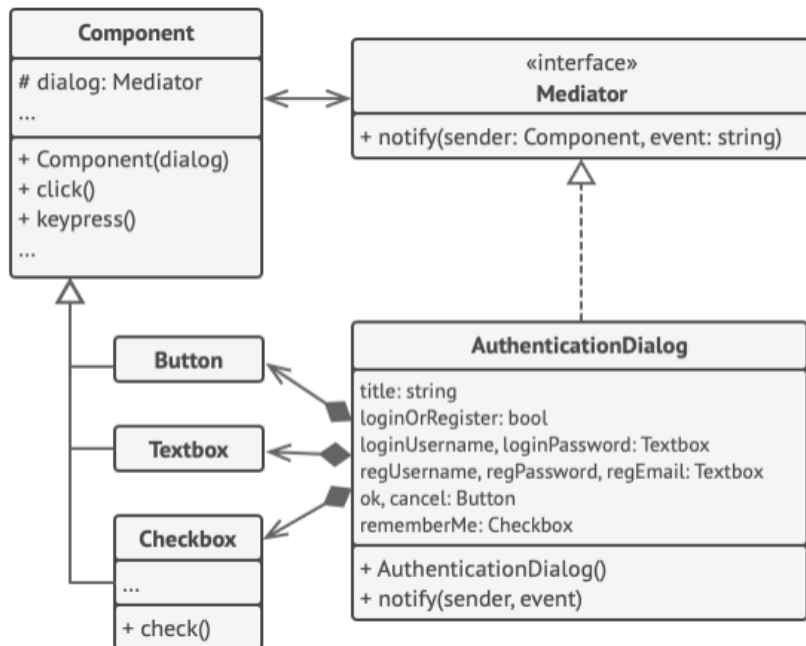
Consecuencias:

- **+** Separación clara de responsabilidades
- **+** Múltiples iteradores sobre misma colección
- **-** Overkill para colecciones simples

4. Mediator (Mediador)



- 1 Componentes que contienen lógica de negocio. Referencia a la interfaz mediadora.
- 2 Interfaz que declara métodos de comunicación con componentes. Normalmente incluyen un único método de notificación.
- 3 Mediadores Concretos encapsulan las relaciones entre varios componentes.
- 4 Los componentes no deben conocer otros componentes. Cada componente sólo debe notificar a la interfaz mediadora. Cuando la mediadora recibe la notificación, debe poder identificar fácilmente al emisor.



Problema que resuelve: Comunicación compleja entre muchos objetos que genera acoplamiento.

Cómo funciona: Centraliza la comunicación entre objetos a través de un mediador.

Ejemplo:

```
java

// Sistema de chat donde usuarios no se comunican directamente
MediadorChat chat = new MediadorChat();

Usuario alice = new Usuario(chat, "Alice");
Usuario bob = new Usuario(chat, "Bob");
Usuario charlie = new Usuario(chat, "Charlie");

chat.registrarUsuario(alice);
chat.registrarUsuario(bob);
chat.registrarUsuario(charlie);

alice.enviarMensaje("Hola a todos!"); // El mediador distribuye
```

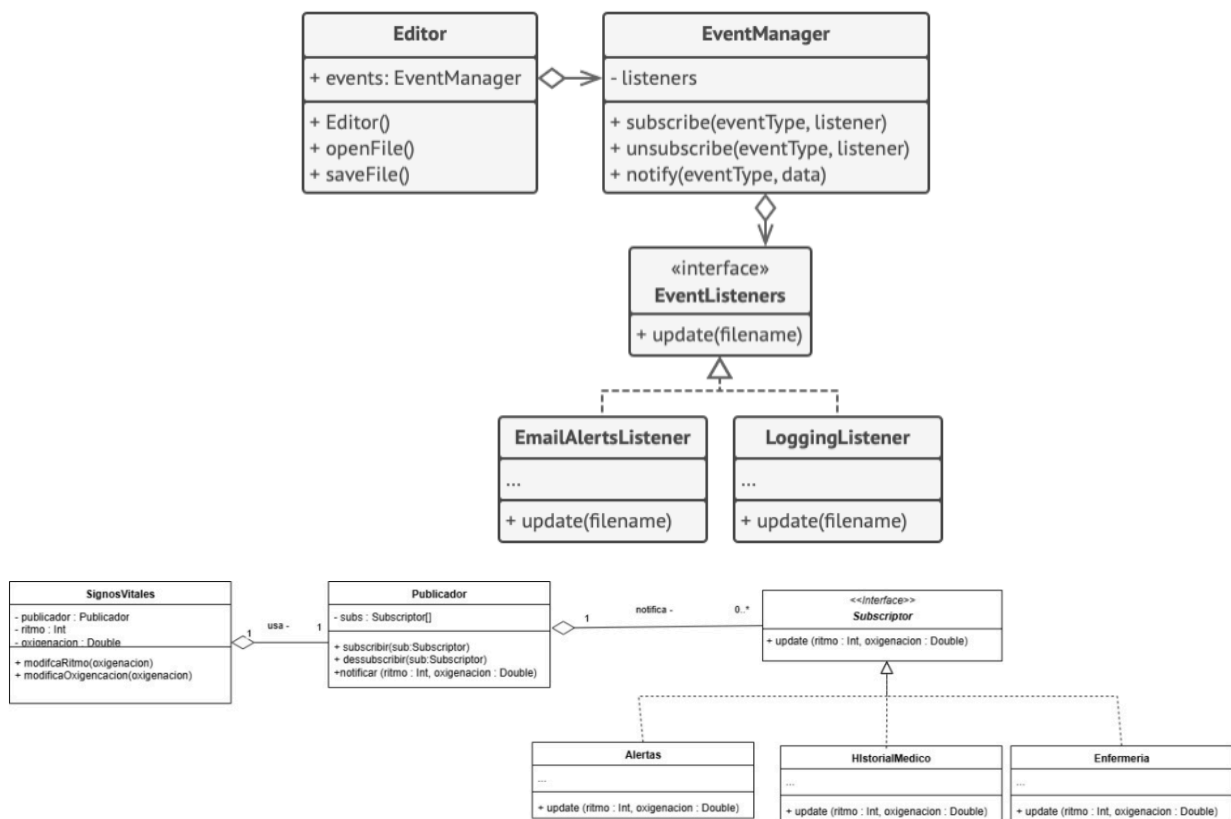
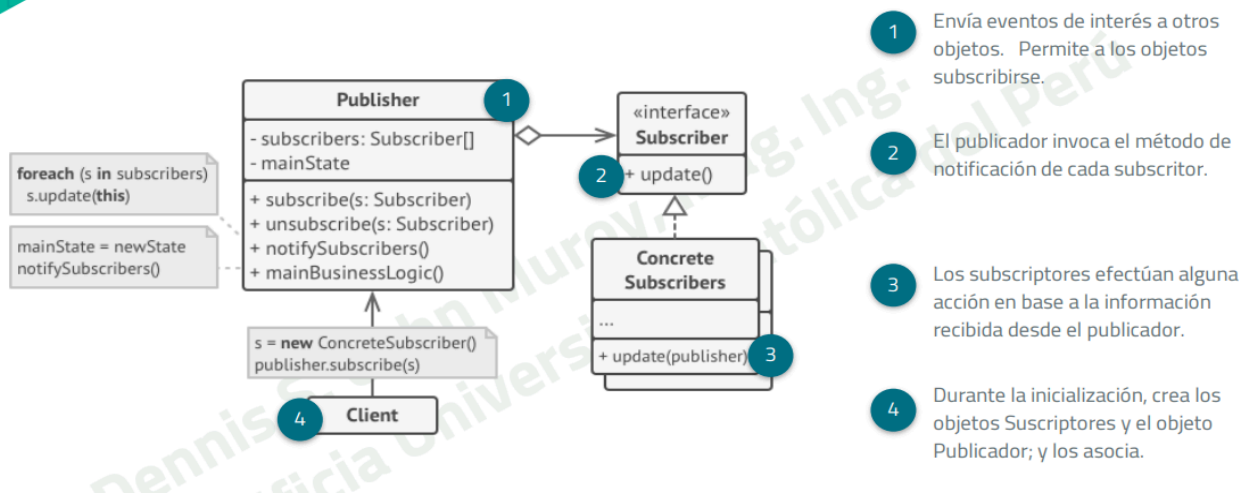
Cuándo usarlo:

-  Muchos objetos se comunican de forma compleja
-  Dependencias caóticas entre objetos
-  Difícil reutilizar componentes por acoplamiento

Consecuencias:

-  Centraliza control en un lugar
-  Reduce acoplamiento entre componentes
-  El mediador puede volverse muy complejo

5. Observer (Observador)



Problema que resuelve: Notificar cambios de estado a múltiples objetos de manera desacoplada.

Cómo funciona: Define una dependencia uno-a-muchos donde cuando un objeto cambia, todos sus dependientes son notificados.

Ejemplo:

java

// Sistema de notificaciones de precios

```
ObservableProducto producto = new Producto("Laptop", 1000);
```

```
Observador usuario1 = new Usuario("Juan");
```

```
Observador usuario2 = new Usuario("Maria");
```

```
Observador sistema = new SistemaNotificaciones();
```

```
producto.agregarObservador(usuario1);
```

```
producto.agregarObservador(usuario2);
```

```
producto.agregarObservador(sistema);
```

```
producto.setPrecio(900); // Todos los observadores son notificados
```

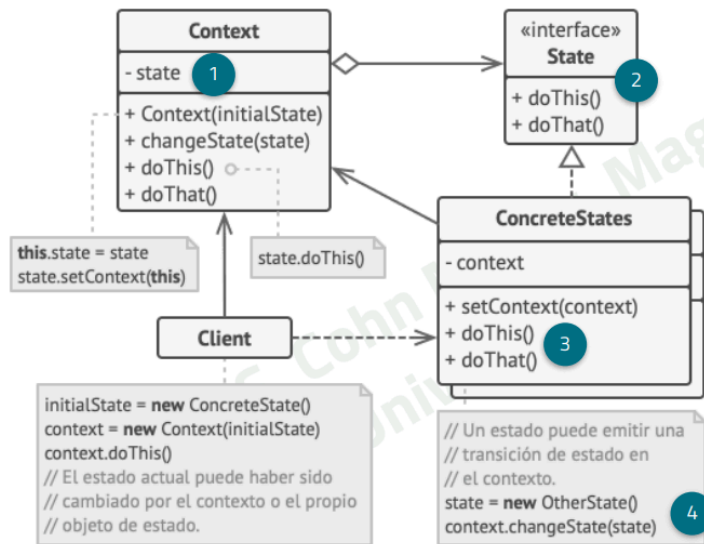
Cuándo usarlo:

-  Cambios en un objeto deben notificarse a otros
-  Relaciones de dependencia dinámicas
-  Broadcasting de cambios

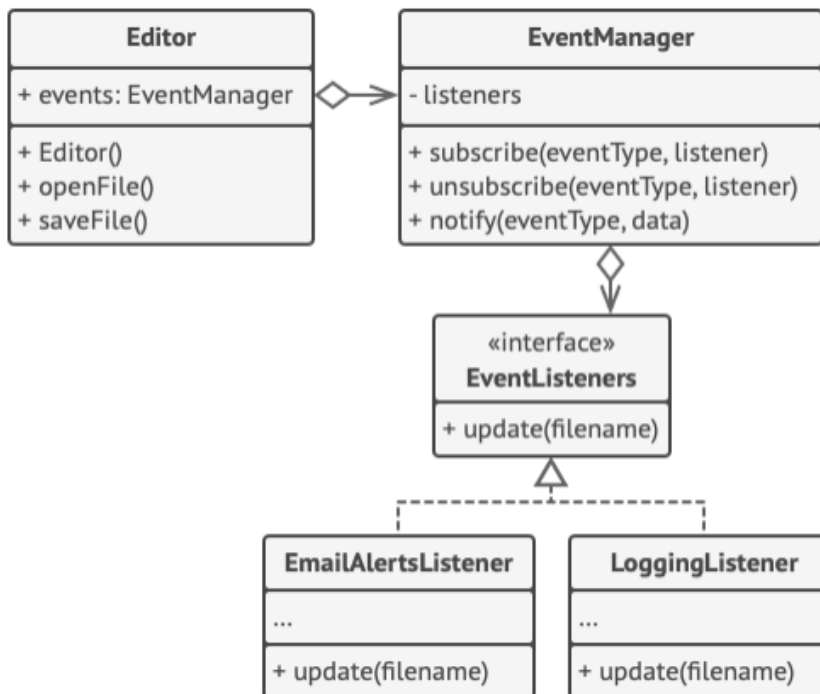
Consecuencias:

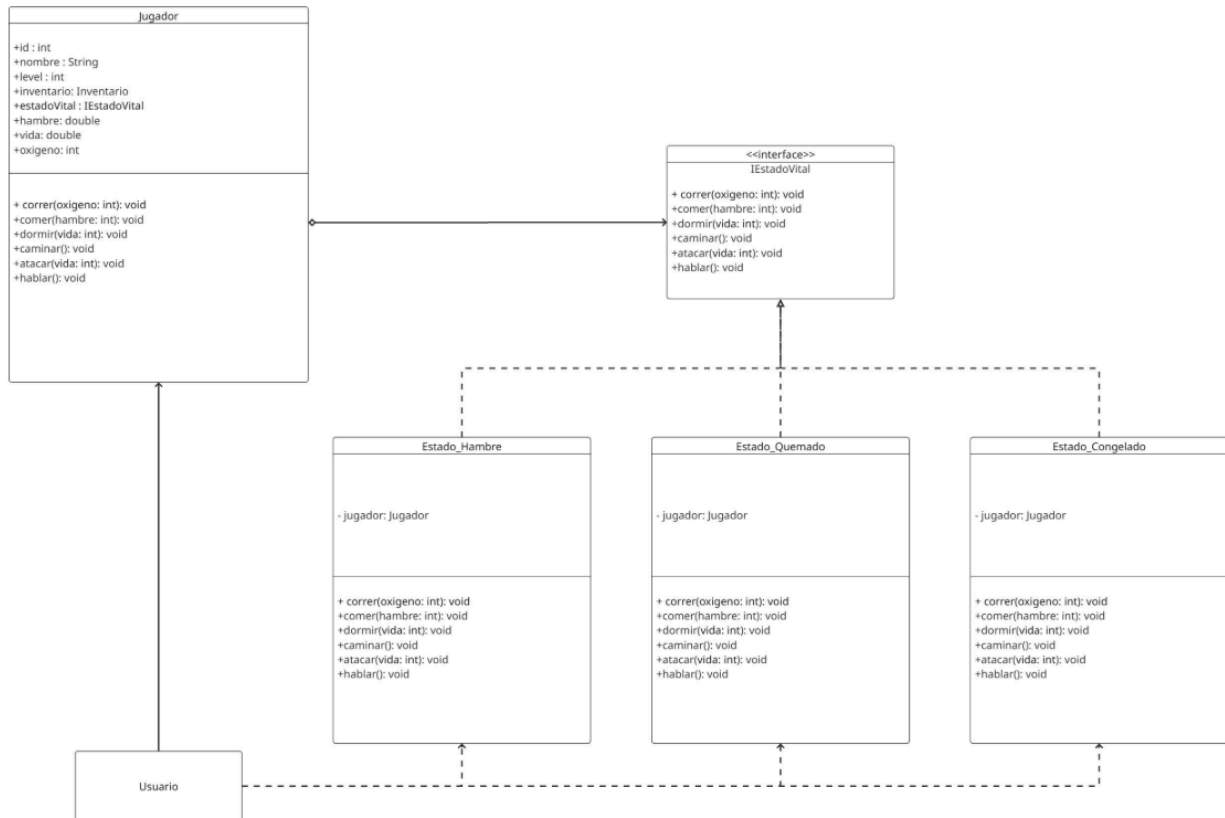
-  Bajo acoplamiento entre sujeto y observadores
 -  Fácil agregar nuevos observadores
 -  Orden de notificación no garantizado
-

6. State (Estado)



- 1 Clase cuyo comportamiento cambia según su estado. Almacena una referencia a un objeto de estado y le delega el trabajo específico del estado
- 2 La interfaz Estado declara los métodos comunes para todos los estados.
- 3 Implementa los métodos específicos del estado. Pueden almacenar una referencia al Contexto para extraer datos requeridos.
- 4 Pueden efectuar un cambio de estado del Contexto.





Problema que resuelve: Comportamiento de un objeto depende de su estado, con muchas condiciones.

Cómo funciona: Permite que un objeto cambie su comportamiento cuando cambia su estado interno.

Ejemplo:

java

```
// Reproductor de música con diferentes estados
```

```
ReproductorContext reproductor = new ReproductorContext();
```

```
reproductor.reproducir(); // Cambia a estado "Reproduciendo"
```

```
reproductor.pausar(); // Cambia a estado "Pausado"
```

```
reproductor.detener(); // Cambia a estado "Detenido"
```

```
// Cada estado tiene diferente comportamiento para los mismos métodos
```

Cuándo usarlo:

-  Comportamiento depende del estado
-  Muchas condiciones que verifican el estado
-  Estados bien definidos y frecuentes

Consecuencias:

-  Elimina condicionales complejas
-  Organiza código por estados
-  Overkill para pocos estados o cambios infrecuentes



Guía Rápida de Selección

Patrón	Usar cuando...	Evitar cuando...
Chain of Responsibility	Múltiples procesadores posibles	El procesador siempre es conocido
Command	Necesitas deshacer/encolar	Operaciones simples y únicas
Iterator	Estructuras de datos complejas	Arrays/listas simples directas
Mediator	Mucha comunicación entre objetos	Pocos objetos, comunicación simple
Observer	Cambios deben notificarse a muchos	Relaciones uno-a-uno fijas
State	Comportamiento depende del estado	Pocos estados o cambios raros

Recomendación: Estos patrones son más útiles en sistemas complejos donde el mantenimiento y extensibilidad son prioritarios.